

# Bayesian World Model for Gene Regulatory Networks

## Technical Specification for Implementation

---

### 1. Project Overview

Build a **particle-based Bayesian world model** that jointly infers gene regulatory network (GRN) structure and predicts interventional expression dynamics. The model maintains uncertainty over both the regulatory graph topology and trajectory predictions simultaneously.

#### Primary outputs:

- $p(G \mid \text{data})$  — posterior distribution over which TF→gene regulatory edges exist
- $p(x_{\{1:T\}} \mid G, \text{do}(\cdot))$  — predicted expression trajectories given graph samples and perturbations
- Calibrated credible intervals over both edge existence and expression predictions

**Key constraint:** These two outputs must be **coupled** — the dynamics model is conditioned on graph samples, and the graph posterior is updated by how well sampled graphs explain observed trajectories.

---

### 2. Datasets

#### Primary: DREAM4 In Silico Network Challenge

**Source:** <https://www.synapse.org/Synapse:syn3049712/wiki/74630> **Download:**

`DREAM4_InSilico_Size10.zip` and `DREAM4_InSilico_Size100.zip`

**Structure per network (5 networks × 2 sizes = 10 networks total):**

| File                                       | Description                                       | Shape                                              | Role                                   |
|--------------------------------------------|---------------------------------------------------|----------------------------------------------------|----------------------------------------|
| <code>*wildtype.tsv</code>                 | Steady-state wild-type expression                 | $1 \times N_{\text{genes}}$                        | Baseline state                         |
| <code>*knockouts.tsv</code>                | Single-gene knockout steady states                | $N_{\text{genes}} \times N_{\text{genes}}$         | Exact $\text{do}(x_k=0)$ interventions |
| <code>*knockdowns.tsv</code>               | Single-gene knockdown steady states               | $N_{\text{genes}} \times N_{\text{genes}}$         | Partial interventions                  |
| <code>*timeseries.tsv</code>               | Expression time courses post-perturbation         | 21 timepoints $\times N_{\text{genes}}$ per series | Core dynamics training signal          |
| <code>*timeseries_perturbations.tsv</code> | Which genes were directly perturbed per series    | $N_{\text{series}} \times N_{\text{genes}}$        | Action/intervention labels             |
| <code>*knockouts_timeseries.tsv</code>     | Trajectory from wildtype to knockout steady state | 21 timepoints per gene                             | Additional interventional trajectories |
| <code>*goldstandard.tsv</code>             | True regulatory edges (0/1)                       | $N_{\text{TF}} \times N_{\text{genes}}$            | Evaluation ground truth                |

### Critical data notes:

- Size10: 10 genes, 5 time series (21 timepoints each), 10 knockout trajectories
- Size100: 100 genes, 10 time series (21 timepoints each), 100 knockout trajectories
- Time series structure: perturbation applied at  $t=0$ , held until  $t=500$ , removed at  $t=500$ , system relaxes to  $t=1000$ . 21 timepoints are sampled from  $[0, 1000]$ .
- No TF subset in DREAM4 — all genes are potential regulators
- Gold standard is in a separate subfolder within the zip: `DREAM4_gold_standards/`

**Combined n/D for Size10:** wildtype (1) + knockouts (10) + knockdowns (10) = 21 samples, 10 genes  $\rightarrow n/D = 2.1$

**Combined n/D for Size100:** wildtype (1) + knockouts (100) + knockdowns (100) = 201 samples, 100 genes  $\rightarrow n/D = 2.01$

**Do NOT use:**

- `Size100_multifactorial` subchallenge — steady-state only, no dynamics
  - `*multifactorial.tsv` steady-state files as primary training data
- 

### 3. Architecture

#### 3.1 Generative Model

The full generative model is:

$$p(G) \cdot p(x_{1:T} \mid G, \text{interventions})$$

**Graph prior  $p(G)$ :**

- Spike-and-slab prior over each edge:  $G_{\{dg\}} \sim \text{Bernoulli}(\pi_{\{dg\}})$
- Global sparsity prior:  $\pi_{\{dg\}} \sim \text{Beta}(1, D-1)$  encouraging  $<1/D$  edges per gene
- Alternative: regularised horseshoe prior on edge weights (continuous relaxation)

**Dynamics model  $p(x_{1:T} \mid G, \text{interventions})$ :**

- Linear SDE (start here):  $dx_i/dt = \sum_j G_{\{ji\}} \cdot w_{\{ji\}} \cdot x_j - \gamma_i \cdot x_i + \sigma_i \cdot \varepsilon_i$
- The interaction matrix is **hard-masked by G** — gene  $j$  can only influence gene  $i$  if  $G_{\{ji\}} = 1$
- Edge weights  $w_{\{ji\}}$  are real-valued (sign = activation/repression, magnitude = strength)
- Degradation rates  $\gamma_i > 0$  per gene, noise scales  $\sigma_i > 0$  per gene

**Intervention handling:**

- Knockout of gene  $k$ : set  $x_k = 0$  and zero out column  $k$  of  $G$  before forward pass — exact  $\text{do}(x_k = 0)$
- Knockdown of gene  $k$ : set  $x_k = 0.5 \cdot x_k^{\text{wildtype}}$  (halve transcription rate)
- Time series perturbation: apply basal activation shifts from `*timeseries_perturbations.tsv` as additive inputs

## 3.2 Inference Model

### Particle-based posterior over $G$ :

- Maintain  $K=20$  graph particles:  $\{G^1, \dots, G^K\}$ , each a binary  $N \times N$  adjacency matrix
- Update via Stein Variational Gradient Descent (SVGD) to approximate  $p(G \mid \text{data})$
- SVGD gradient for particle  $k$ :  $\nabla_{\{G^k\}} \log p(G^k \mid \text{data}) + \text{repulsion from other particles}$
- Repulsion kernel: RBF kernel on vectorised adjacency matrices

### Per-particle dynamics fitting:

- For each  $G^k$ , fit edge weights  $W^k$ , degradation rates  $\gamma^k$ , noise scales  $\sigma^k$  via PyMC
- Priors:  $w_{\{ji\}} \mid G_{\{ji\}}=1 \sim \text{Normal}(0, \sigma_w)$ ,  $\gamma_i \sim \text{HalfNormal}(1)$ ,  $\sigma_i \sim \text{HalfNormal}(0.5)$
- Likelihood: Gaussian process over time series trajectories given ODE solution

## 3.3 Predictive Distribution

For a novel perturbation  $\text{do}(\cdot)$ :

```
python
```

$$p(x_{\{1:T\}} \mid \text{do}(\cdot)) \approx (1/K) \sum_k p(x_{\{1:T\}} \mid G^k, W^k, \text{do}(\cdot))$$

- Sample  $K$  trajectories, one per particle
- Output: mean trajectory + credible intervals across particles
- Structural uncertainty (which graph is correct) and dynamical uncertainty (given the graph) are both represented

---

## 4. PyMC Implementation

### 4.1 Per-Particle Model

For a fixed graph  $G^k$ , fit the dynamics parameters with PyMC:

```
python
```

```

import pymc as pm
import numpy as np
from scipy.integrate import odeint

def build_per_particle_model(G_k, timeseries_data, perturbation_data, knockout_data)
    """
    G_k: binary adjacency matrix (N_genes × N_genes)
    timeseries_data: list of (timepoints, expression_matrix) tuples
    perturbation_data: basal activation shifts per time series
    knockout_data: list of (knocked_out_gene_idx, steady_state_expression) tuples
    """
    N = G_k.shape[0]
    active_edges = np.argwhere(G_k) # Only fit weights for active edges

    with pm.Model() as model:
        # Edge weights – only for active edges in G_k
        w_flat = pm.Normal("w", mu=0, sigma=1, shape=len(active_edges))
        W = pm.Deterministic("W", build_weight_matrix(w_flat, active_edges, N))

        # Degradation rates (must be positive)
        gamma = pm.HalfNormal("gamma", sigma=1, shape=N)

        # Noise scale
        sigma_obs = pm.HalfNormal("sigma_obs", sigma=0.5, shape=N)

        # ODE forward pass for each time series
        for i, (timepoints, expression, perturbation) in enumerate(timeseries_data):
            x_pred = pm.Deterministic(
                f"x_pred_{i}",
                ode_solve(W, gamma, G_k, timepoints, expression[0], perturbation)
            )
            pm.Normal(f"obs_{i}", mu=x_pred, sigma=sigma_obs, observed=expression)

        # Knockout likelihood – exact do-interventions
        for gene_idx, ko_expression in knockout_data:
            G_ko = apply_knockout(G_k, gene_idx) # Zero column gene_idx
            x_ko_pred = pm.Deterministic(
                f"x_ko_{gene_idx}",
                ode_steady_state(W, gamma, G_ko)
            )
            pm.Normal(f"ko_obs_{gene_idx}", mu=x_ko_pred, sigma=sigma_obs,
                observed=ko_expression)

```

```
return model
```

## 4.2 NUTS Sampling (Size10 Validation)

For DREAM4 Size10 networks (10 genes), run NUTS as ground truth:

```
python
```

```
def sample_nuts(model, draws=1000, tune=500, chains=4):  
    with model:  
        trace = pm.sample(  
            draws=draws,  
            tune=tune,  
            chains=chains,  
            target_accept=0.9,  
            nuts_sampler="numpyro", # Faster than default  
            return_inferencedata=True  
        )  
    return trace
```

## 4.3 ArviZ Diagnostics

After sampling, use ArviZ for convergence and posterior analysis:

```
python
```

```

import arviz as az

def diagnose_trace(trace, model_name):
    # Convergence diagnostics
    r_hat = az.rhat(trace)
    ess = az.ess(trace)

    # All R-hat < 1.01 and ESS > 100 required
    assert (r_hat.to_array() < 1.01).all(), "R-hat convergence failure"
    assert (ess.to_array() > 100).all(), "Insufficient effective samples"

    # Plot posterior distributions over edge weights
    az.plot_posterior(trace, var_names=["W", "gamma"])

    # Plot energy (NUTS health check)
    az.plot_energy(trace)

    # Posterior predictive checks
    with model:
        ppc = pm.sample_posterior_predictive(trace)
    az.plot_ppc(az.from_pymc(posterior_predictive=ppc))

    return {
        "r_hat_max": float(r_hat.to_array().max()),
        "ess_min": float(ess.to_array().min()),
        "trace": trace
    }

```

## 4.4 Graph Posterior via SVGD

python

```

import torch
import torch.nn.functional as F

class SVGDGraphParticles:
    def __init__(self, K, N_genes, prior_sparsity=0.1):
        """
        K: number of particles
        N_genes: number of genes (adjacency matrix size)
        prior_sparsity: expected fraction of true edges
        """
        self.K = K
        self.N = N_genes
        # Initialise logits from prior – sparse initialisation
        self.logits = torch.randn(K, N_genes, N_genes) - 2.0 # Biased toward 0 (no
        self.logits.requires_grad_(True)

    def sample_graphs(self, hard=False, temperature=0.5):
        """Sample binary graphs via Gumbel-Softmax or hard threshold"""
        if hard:
            return (torch.sigmoid(self.logits) > 0.5).float()
        else:
            return torch.sigmoid((self.logits + gumbel_noise(self.logits)) / tempera

    def svgd_update(self, log_prob_grads, bandwidth=1.0):
        """
        log_prob_grads: gradient of log p(G^k | data) for each particle k
        Updates logits via SVGD
        """
        logit_matrix = self.logits.view(self.K, -1)

        # RBF kernel between particles
        pairwise_dists = torch.cdist(logit_matrix, logit_matrix)
        h = bandwidth * torch.median(pairwise_dists)**2 / np.log(self.K)
        K_matrix = torch.exp(-pairwise_dists**2 / h)

        # SVGD gradient: attraction (likelihood) + repulsion (diversity)
        grad_matrix = log_prob_grads.view(self.K, -1)
        svgd_grad = (K_matrix @ grad_matrix - torch.autograd.grad(
            K_matrix.sum(), logit_matrix)[0]) / self.K

        self.logits.grad = -svgd_grad.view(self.K, self.N, self.N)

    def edge_probabilities(self):

```



```
        """Marginal edge probability across particles – used for AUPR"""
        return torch.sigmoid(self.logits).mean(dim=0).detach().numpy()

    def structural_uncertainty(self):
        """Variance of edge probabilities across particles"""
        probs = torch.sigmoid(self.logits)
        return probs.var(dim=0).detach().numpy()
```

---

## 5. ODE Dynamics Model

### 5.1 Linear SDE (start here)

```
python
```

```

from scipy.integrate import solve_ivp
import numpy as np

def ode_rhs(t, x, W, gamma, perturbation, t_perturb_end=500.0):
    """
    Right-hand side of the gene regulatory ODE.
    W: weight matrix (N × N), masked by graph G
    gamma: degradation rates (N,)
    perturbation: basal activation shifts (N,) – nonzero for directly perturbed genes
    t_perturb_end: time at which perturbation is removed (500 in DREAM4)
    """
    basal = perturbation if t < t_perturb_end else np.zeros_like(perturbation)
    return W @ x - gamma * x + basal

def ode_solve(W, gamma, timepoints, x0, perturbation, t_perturb_end=500.0):
    """
    Solve ODE and return expression at observed timepoints.
    Returns array of shape (len(timepoints), N_genes)
    """
    sol = solve_ivp(
        fun=lambda t, x: ode_rhs(t, x, W, gamma, perturbation, t_perturb_end),
        t_span=(timepoints[0], timepoints[-1]),
        y0=x0,
        t_eval=timepoints,
        method='RK45',
        rtol=1e-6,
        atol=1e-8
    )
    return sol.y.T # (timepoints, N_genes)

def apply_knockout(G, gene_idx):
    """Apply do(x_{gene_idx} = 0): zero out column gene_idx of G"""
    G_ko = G.copy()
    G_ko[:, gene_idx] = 0 # No gene can be regulated by knocked-out gene
    G_ko[gene_idx, :] = 0 # Knocked-out gene has no output
    return G_ko

```

## 5.2 Pytensor-Compatible ODE for PyMC

For gradient-based sampling in PyMC, use `pytensor` ops:

```
python
```

```

import pytensor.tensor as pt
from pytensor.graph.op import Op

class ODEOp(Op):
    """
    Custom PyTensor Op wrapping scipy ODE solver.
    Required for PyMC to differentiate through the ODE.
    """
    itypes = [pt.dmatrix, pt.dvector, pt.dvector, pt.dvector] # W, gamma, x0, pertu
    otypes = [pt.dmatrix] # predicted trajectories

    def __init__(self, timepoints, G_mask):
        self.timepoints = timepoints
        self.G_mask = G_mask

    def perform(self, node, inputs, outputs):
        W, gamma, x0, perturbation = inputs
        W_masked = W * self.G_mask # Apply graph mask
        traj = ode_solve(W_masked, gamma, self.timepoints, x0, perturbation)
        outputs[0][0] = traj

    def grad(self, inputs, output_grads):
        # Finite difference gradient – replace with adjoint method for speed
        raise NotImplementedError("Use pm.ode.DifferentialEquation instead")

```

**Recommended:** Use `pm.ode.DifferentialEquation` which handles gradients automatically:

```

python

def grn_ode(x, t, p, G_mask):
    W = p[:N*N].reshape(N, N) * G_mask
    gamma = p[N*N:N*N+N]
    perturbation = p[N*N+N:]
    return W @ x - gamma * x + perturbation

# Register with PyMC
ode_model = pm.ode.DifferentialEquation(
    func=grn_ode,
    times=timepoints,
    n_states=N_genes,
    n_theta=N_genes*N_genes + N_genes + N_genes, # W flat + gamma + perturbation
)

```

## 6. Evaluation Protocol

### 6.1 Graph Recovery (AUPR)

python

```
from sklearn.metrics import average_precision_score
import arviz as az
import numpy as np

def evaluate_graph_recovery(particles, gold_standard_file):
    """
    particles: SVGDGraphParticles object
    gold_standard_file: path to *goldstandard.tsv
    Returns AUPR against gold standard edges
    """
    # Load gold standard
    gold = np.loadtxt(gold_standard_file, delimiter='\t', skiprows=1)
    true_edges = gold[:, 2].astype(int) # Third column is 0/1

    # Edge probabilities from particle ensemble
    edge_probs = particles.edge_probabilities().flatten()

    aupr = average_precision_score(true_edges, edge_probs)
    return aupr

def evaluate_nuts_graph_posterior(trace, gold_standard):
    """For Size10: compare NUTS posterior to gold standard"""
    # Posterior edge inclusion probabilities from NUTS
    # Requires indicator variables in the model
    edge_probs = trace.posterior["edge_indicators"].mean(dim=["chain", "draw"]).values
    aupr = average_precision_score(gold_standard.flatten(), edge_probs.flatten())
    return aupr
```

### 6.2 Predictive Calibration (ArviZ)

python

```

def evaluate_predictive_calibration(particles, per_particle_traces,
                                   held_out_trajectories):
    """
    Evaluate whether predictive intervals are calibrated.
    Uses ArviZ's coverage diagnostic.
    """
    all_predictions = []

    for k, trace_k in enumerate(per_particle_traces):
        with per_particle_models[k]:
            ppc_k = pm.sample_posterior_predictive(trace_k)
            all_predictions.append(ppc_k)

    # Combine across particles (structural + dynamical uncertainty)
    combined_ppc = np.concatenate([p["obs_0"] for p in all_predictions], axis=0)

    # ArviZ calibration: check that 90% CI contains 90% of observations
    coverage = az.plot_bpv(
        az.from_dict(posterior_predictive={"x": combined_ppc},
                    observed_data={"x": held_out_trajectories})
    )

    return coverage

```

### 6.3 NUTS vs Particles Comparison (Size10 Only)

This is the key validation. On Size10 networks, NUTS gives exact posterior — compare particle approximation against it:

```
python
```

```
def compare_nuts_vs_particles(nuts_trace, particles, variable="W"):
    """
    Compare marginal posteriors from NUTS vs particle ensemble.
    Uses ArviZ's comparison tools.
    """
    # NUTS posterior
    nuts_post = az.extract(nuts_trace, var_names=[variable])

    # Particle posterior (treat per-particle MCMC samples as a mixture)
    particle_samples = np.concatenate([
        az.extract(t, var_names=[variable]) for t in particle_traces
    ], axis=-1)

    # Compare with energy distance or MMD
    az.plot_dist_comparison(nuts_post, particle_samples)

    # R-hat across particles (should be ~1 if particles have converged)
    combined = az.concat([t for t in particle_traces], dim="chain")
    return az.rhat(combined)
```

## 7. Implementation Phases

### Phase 1 — Data Pipeline (start here)

1. Download DREAM4 Size10 zip from Synapse
2. Parse `*timeseries.tsv`, `*timeseries_perturbations.tsv`, `*knockouts.tsv`, `*knockouts_timeseries.tsv`, `*goldstandard.tsv` for all 5 networks
3. Build data objects: `(timepoints, trajectories, perturbations, knockout_pairs, gold_edges)`
4. Validate: plot a few time series, confirm shapes match spec above

### Phase 2 — Fixed-Graph Baseline

1. Fix `G = gold_standard` (use the true graph)
2. Fit dynamics model with PyMC NUTS — this is the performance ceiling
3. Validate convergence with ArviZ (R-hat < 1.01, ESS > 100)
4. Confirm predictive trajectories match held-out time series

Phase 3 — NUTS Over Full Model (Size10)

- 1. Add graph structure as a variable (spike-and-slab Bernoulli per edge)
- 2. Sample jointly over G and dynamics parameters with NUTS
- 3. This is computationally expensive but valid for 10-gene networks
- 4. Use ArviZ to compute edge inclusion probabilities → AUPR
- 5. This is the **ground truth posterior** against which particles are validated

Phase 4 — Particle-Based Inference

- 1. Implement `SVGDGraphParticles` with K=20 for Size10
- 2. For each particle, fit dynamics per Phase 2 approach
- 3. SVGD update step after each dynamics fit
- 4. Validate: particle edge probabilities should converge to NUTS posteriors from Phase 3
- 5. Compare AUPR: particles vs NUTS vs GENIE3 baseline

Phase 5 — Scale to Size100

- 1. Replace NUTS with particles only (NUTS intractable at 100 genes)
- 2. K=20-50 particles depending on compute budget
- 3. Evaluate AUPR on all 5 Size100 networks
- 4. Compare against GENIE3 and GCV-ridge baselines from Paper 1

8. Baselines to Implement

These are required for comparison:

| Baseline           | Implementation                              | Purpose                            |
|--------------------|---------------------------------------------|------------------------------------|
| GENIE3             | <code>pip install arboreto</code>           | Standard benchmark                 |
| GCV-Ridge          | Analytical (SVD + GCV), see Paper 1         | Paper 1 best method                |
| Fixed-graph NUTS   | Phase 2 above                               | Performance ceiling                |
| Observational-only | Run model on wildtype + multifactorial only | Quantify interventional data value |

## 9. Dependencies

```
txt

pymc>=5.0
arviz>=0.17
pytensor>=2.18
numpy>=1.24
scipy>=1.10
scikit-learn>=1.3
pandas>=2.0
matplotlib>=3.7
torch>=2.0          # For SVGD particle updates
arboreto            # GENIE3 baseline
numpyro>=0.13       # Faster NUTS sampler backend for PyMC
jax>=0.4            # Required by numpyro
```

## 10. Directory Structure

```
grn_world_model/
├─ data/
│   ├─ raw/                # DREAM4 zip files go here
│   └─ processed/          # Parsed numpy arrays per network
├─ src/
│   ├─ data_loader.py      # DREAM4 parsing and preprocessing
│   ├─ ode_model.py        # ODE RHS, solve_ivp wrapper, knockout logic
│   ├─ pymc_model.py       # Per-particle PyMC model definition
│   ├─ svgd_particles.py   # SVGDGraphParticles class
│   └─ inference.py        # Full training loop: SVGD + per-particle
NUTS/ADVI
├─ evaluation.py           # AUPR, calibration, NUTS vs particles
comparison
├─ baselines.py            # GENIE3, GCV-ridge
├─ notebooks/
│   ├─ 01_data_exploration.ipynb  # Visualise DREAM4 time series
│   ├─ 02_fixed_graph_baseline.ipynb
│   ├─ 03_nuts_full_model.ipynb   # Size10 ground truth
│   └─ 04_particles_size100.ipynb
└─ results/
```



## 11. Key Design Constraints

These come directly from the failure mode analysis in Paper 1 and must be respected:

1. **No ELBO on observational data alone.** The likelihood must include knockout trajectories as interventional terms. Wildtype expression alone is insufficient — it cannot separate regulatory signal from environmental confounding.
2. **No dense aggregation.** The graph  $G$  must explicitly gate information flow in the dynamics model. Gene  $j$  can only influence gene  $i$  if  $G_{\{ji\}} = 1$ . Do not allow mean-pool or global attention over all genes.
3. **No mean-field VI over horseshoe.** For Size10, use NUTS. For Size100, use the particle ensemble (each particle is a point estimate of  $G$ , but the ensemble represents the posterior). Do not use mean-field Gaussian VI — it collapses horseshoe to ridge.
4. **Interventions are exact do-calculus, not instruments.** DREAM4 knockouts are  $\overline{P(Y \mid \text{do}(X=0))}$ . Implement them as hard zeros on the relevant gene column/row in  $G$ , not as approximate IV corrections.

## 12. Expected Results

Based on Paper 1 results and DREAM4 structure:

- **Fixed-graph NUTS (ceiling):** Should achieve near-perfect trajectory prediction when true  $G$  is given
- **NUTS full model (Size10):** AUPR > 0.5 expected given exact interventional data (vs ~0.25 GENIE3 on DREAM5 observational)
- **Particles (Size10):** Should approach NUTS AUPR within 5-10% if  $K=20$  is sufficient
- **Particles (Size100):** Comparable to or exceeding GENIE3 as primary target

The key empirical contribution is showing that interventional data (knockouts) produces substantially better calibrated posteriors than observational data alone — this directly validates the IV framing from Paper 1 Section 5 with actual ground truth.